

Stage 2.5 Concierge Pod Claiming — Implementation Report

Summary

Stage 2.5 is implemented. Platform admins can create ownerless, menu-only pods from Admin Pods, attach Stage 2 unclaimed vendors and publish menus, then invite a verified matching account to claim the existing pod. Claim acceptance adds or promotes the claimant's `PodMembership` to `owner` without duplicating the pod or mutating vendors, menus, QR/public URLs, ordering intent, routing, or payments.

This completes the concierge onboarding model for the orderless launch: Open Order can create and prepare pod value before asking the pod owner to create an account.

Existing Ownership Architecture

Concept	Implementation
Source of truth	<code>PodMembership</code> with <code>PodMembershipRole.owner</code> <code>manager</code>
<code>Pod.ownerId</code>	Does not exist (only contact fields)
Claimed	Derived: ≥ 1 owner membership
Self-service creation	<code>createPodProfile</code> still creates an owner membership
<code>PodVendorInvite</code>	Unchanged; attaches vendors to pods, never grants pod ownership

Stage 2.5 mirrors Stage 2 vendor claiming with a dedicated `PodClaimInvite` rather than overloading `PodVendorInvite`.

Schema / Migration

Added `PodClaimInvite` :

- unique `podId` (one lifecycle row per pod)
- unique `tokenHash`
- invited email, expiry, claim, revoke, sent timestamps
- inviter / claimant user relations

- cascade delete with pod; `SetNull` on users

Migration:

```
prisma/migrations/20260905043000_pod_claim_invite/migration.sql
```

- Additive and non-destructive
- Does not update existing pods, memberships, vendors, menus, or ordering flags
- Rollback: drop `PodClaimInvite`

Connected database status at implementation time: pending (not deployed, to avoid mutating the shared environment). Stage 2's vendor claim migration may also still be pending depending on environment.

Concierge Pod Creation

`adminCreateUnclaimedPod` / `adminCreateUnclaimedPodAction` :

- Admin Pods “Create pod” form (name, address, optional contact, optional owner email, admin reason)
- Validates / normalizes input
- Generates slug via `uniquePodSlugFromName`
- Detects normalized name (+ address when present) duplicates; requires explicit override
- Creates `Pod` with `orderingEnabled: false` explicitly (schema default remains `true`)
- Creates no user and no `PodMembership`
- Audits `UNCLAIMED_POD_CREATED`
- Revalidates admin and public surfaces

Self-service `createPodProfile` is unchanged.

Claim-State Derivation

`getPodClaimState` derives:

State	Condition
<code>unclaimed</code>	No owner membership; no active invite
<code>invite_pending</code>	Active non-revoked invite not expired
<code>invite_expired</code>	Active invite past expiry
<code>claimed</code>	≥ 1 owner membership

Manager memberships do not count. Revoked invites return to `unclaimed` (no permanent revoked product state).

Token Security

Reuses Stage 2 secure invite infrastructure:

- 32-byte base64url token generation
- SHA-256 hash-only persistence
- Seven-day TTL (`POD_CLAIM_INVITE_TTL_MS`)
- Resend rotates token and invalidates old links
- Revoke sets `revokedAt`
- Raw token returned once to admin; never audited or logged in email dry-run previews (`sensitiveContent: true`)
- Dedicated URL: `/claim/pod/[token]`

Authentication Return Flow

Extended Stage 2 safe return handling via `isOwnershipClaimPath` (vendor + pod):

- Post-login pending-setup bypass returns directly to claim path
- Registration preserves claim return and skips forcing a second pod setup
- Email verification stores sanitized claim `next` in token metadata/URL
- Verify page shows "Continue to claim pod" or "Continue to claim vendor"
- Claimant must be authenticated, enabled, email-verified, and match invited email

Claim Transaction

`acceptPodClaimInvite` uses a serializable transaction with retry:

1. Re-read user and invite
2. Validate lifecycle, email match, verified/enabled account
3. Ensure zero owner memberships
4. Atomically consume invite (`updateMany`)
5. Upsert claimant `PodMembership` to owner
6. Set `registrationIntent: pod_owner` , clear role selection
7. Audit `POD_CLAIMED` after commit
8. Redirect to `/pod/[podId]/dashboard?claimed=1`

Simultaneous claims yield exactly one owner.

Admin UI

- **Admin Pods:** Create pod form; Claimed/Unclaimed ownership filter; ownership column
- **Admin Pod Detail:** Ownership status card + Ownership section with send / resend / revoke
- Claim actions remain platform-admin only
- Manual user-id owner attach in Advanced remains available but separate from claim invite flow

Pod Owner First Login

After claim:

- Redirect to existing pod dashboard with `?claimed=1`
- One concise success message
- Existing membership authorization grants dashboard, settings, vendor roster, QR/sharing tools
- No onboarding wizard, payment prompt, or vendor auto-invite

Public Visibility

Claim state is admin-only. Unclaimed pods may remain public and menu-only when readiness permits. Customer surfaces do not show Unclaimed / Claim this pod / Owner not registered.

Vendor claim state remains independent of pod claim.

Data Preservation

Claim acceptance writes only:

- `PodClaimInvite` lifecycle fields
- one `PodMembership` create/promote
- claimant registration fields
- `AdminAuditLog`

Does not alter pod id/slug/QR destination, `orderingEnabled`, vendors, `PodVendor` rows, menus, hours, branding, routing, Stripe/Square/Deliverect, or orders.

Tests

Focused Stage 2.5 + related auth suite: **9 files, 91 passed.**

Coverage includes:

- concierge creation without owner/user
- duplicate warning / override
- claim-state derivation (including manager non-claim)
- token hash / expiry / rotate / revoke
- wrong account, unverified, expired, revoked, already claimed
- atomic double-claim concurrency
- preservation assertions
- admin authorization contracts
- public invisibility
- auth return path for pod claim
- dashboard success banner

Production `npm run build` : **passed** (includes `/claim/pod/[token]`).

`npx prisma validate / generate` : **passed**.

Full repository suite baseline failures remain outside this stage (same class of pre-existing mock/UI contract issues noted in Stage 2). Stage 2.5 did not introduce focused-suite regressions.

Files Changed

Principal additions/updates:

- `prisma/schema.prisma` ,
`prisma/migrations/20260905043000_pod_claim_invite/`
- `src/services/admin-concierge-pod.service.ts`
- `src/services/pod-claim-invite.service.ts`
- `src/lib/pod-claim-state.ts`
- `src/lib/auth/pod-claim-path.ts` , `ownership-claim-path.ts`
- `src/lib/email/pod-claim-invite-email.ts`
- `src/actions/pod-claim.actions.ts` , `admin-pod.actions.ts`
- `src/app/claim/pod/[token]/`
- `src/app/admin/(dashboard)/pods/` (create form, search filter, ownership UI)
- `src/app/pod/[podId]/dashboard/page.tsx`
- Auth: `post-login-destination.ts` , `RegisterForm.tsx` , `email-verification.service.ts` , `verify-email/page.tsx`
- Tests under `src/lib/*pod-claim*` , `src/services/*pod-claim*` ,
`src/lib/concierge-pod-claiming-surfaces.test.ts`

Deferred Work

- Engagement analytics
- Self-service pod creation expansion
- Pod-owner vendor creation
- Bulk onboarding / import
- AI menu import
- Ownership transfer / multi-owner management beyond existing admin tools
- Pod-owner ordering-mode controls if still admin-only
- Deploy pending claim migrations and live browser smoke test

Manual QA Required

After deploying `20260905043000_pod_claim_invite` (and Stage 2 vendor claim migration if still pending):

1. Admin creates unclaimed pod → Ownership Unclaimed, menu-only, no membership
2. Add unclaimed vendors, publish menus, confirm public pod preview works before claim
3. Send / resend / revoke claim invite; confirm old links fail
4. Existing verified invited account claims successfully → dashboard `?claimed=1`
5. New account: register → verify → Continue to claim pod → claim succeeds; no second pod
6. Wrong email rejected
7. Before/after: same pod id, slug, QR/public URL, vendors, menus, PodVendor IDs, orderingEnabled
8. Vendor claim states unchanged by pod claim